

Project 2 - Dynamic Web Server

Start Assignment

- Due Oct 30 by 10pm
- Points 50
- Submitting a text entry box or a website url
- Available Oct 15 at 10:55am - Oct 30 at 10pm

Assignment 2

Dynamic Web Server

Task

Create a dynamic web server that leverages server-side rendering for a data set of your choice relating to some aspect of sustainability. Your server will serve static files (e.g. CSS, JavaScript, Jpeg, and Png files) as well as dynamic routes for pages relating to your chosen data set.

NOTE: you are allowed to use any Node.js modules (built-in, installed via npm, or written yourself) to help develop your dynamic web server. You are also allowed to use any CSS / client-side JavaScript libraries to help design your web pages.

About the Data Sets

You will select a data set from one of the two following repositories: [FiveThirtyEight](https://github.com/fivethirtyeight/data)  (<https://github.com/fivethirtyeight/data>) or [Awesome Public Datasets](https://github.com/awesomedata/awesome-public-datasets)  (<https://github.com/awesomedata/awesome-public-datasets>) that has some connection to sustainability. You must get the selected data set approved by the instructor prior to beginning implementation of your dynamic web server.

Once you have selected an appropriate data set, you will need to download the data and store it in an SQLite3 database. Note: I can help with this step (as the focus of this project is web development not database design).

Dynamic Web Server (50 pts)

To earn 38/50 points (grade: C)

- Package.json
 - Fill out the author and contributors sections in package.json (author should be whoever's GitHub account is used to host the code, contributors should be all group members)
 - Fill out the URL of the repository
 - Ensure all used modules downloaded via NPM are in the dependencies object
 - Ensure the "node_modules" folder is **not** included in the GitHub repository
- Dynamic Web Pages
 - Create at least 3 dynamic routes for viewing the data from different points of view (for example, if choosing a data set on energy consumption, the routes may be to view the data by state, by year, or by energy type)
 - Write HTML template files that data can be inserted into
 - Include appropriate representation of the data
 - Should include some text-based data (e.g. headers, paragraphs, tables, etc.) - dynamically populate text content

- Should include some visual based data (e.g. images, videos, etc.) - dynamically populate src and alt
- Site Navigation
 - Create a standard navigation to enable a user to navigate between dynamic routes
 - Create a home page (OK to select one dynamic route as the default home page)

To earn a grade of A or B

- 4 pts: create dynamically populated 'previous' and 'next' links in HTML template files that link to the previous or next page for its data type
 - Link can either be disabled or circle around when at the first/last item
- 4 pts: send a proper 404 error if a requested source does not exist in the database
 - Can be plain text, but should be customized to the request (e.g. "Error: no data for state FB", or "Error: no data for year 2030")
- 4 pts: create a chart/graph to visualize the data
 - Tip: I would suggest using an existing library if generating the graphs on your HTML pages. Some possible choices are:
 - Plotly (<https://plotly.com/javascript/> ↗ [\(https://plotly.com/javascript/\)](https://plotly.com/javascript/))
 - ChartJS (<https://www.chartjs.org/> ↗ [\(https://www.chartjs.org/\)](https://www.chartjs.org/))
 - AnyChart (<https://www.anychart.com/> ↗ [\(https://www.anychart.com/\)](https://www.anychart.com/))
 - CanvasJS (<https://canvasjs.com/> ↗ [\(https://canvasjs.com/\)](https://canvasjs.com/))

Starter Code

Starter code can be found on GitHub: <https://github.com/tmarrinan/webdev-dynamic> ↗ [\(https://github.com/tmarrinan/webdev-dynamic\)](https://github.com/tmarrinan/webdev-dynamic). This repo contains a basic skeleton for the project as well as starter / example code for the server. Foundation CSS / JS files are also included in the public web directory, so you can easily use that when laying out the content for your web pages.

IMPORTANT: One group member should **fork** this repo, then invite other group members as collaborators.

Note: the server will use the sqlite3 NPM module to interface with our database. The API for this module can be found on their GitHub page: <https://github.com/TryGhost/node-sqlite3/wiki/API> ↗ [\(https://github.com/TryGhost/node-sqlite3/wiki/API\)](https://github.com/TryGhost/node-sqlite3/wiki/API). Of particular interest will be the ``all(sql, [param, ...], [callback])`` ↗ [\(https://github.com/TryGhost/node-sqlite3/wiki/API#allsql-param---callback\)](https://github.com/TryGhost/node-sqlite3/wiki/API#allsql-param---callback) method. Also look at the ``run(sql, [param, ...], [callback])`` ↗ [\(https://github.com/TryGhost/node-sqlite3/wiki/API#runsql-param---callback\)](https://github.com/TryGhost/node-sqlite3/wiki/API#runsql-param---callback) method to see how to use the param optional parameter when constructing your queries.

Submission

Code should be saved in a repository on GitHub. Do NOT add your node_modules directory to your repository. This is what package.json is for - it will store which modules you use for your project. We will hopefully deploy our servers on render.com ↗ [\(https://render.com\)](https://render.com).

In order to submit, you should enter the the project's GitHub repo URL in Canvase. If we get to it, please also submit your live render.com URL as a comment.

IMPORTANT: Only one group member needs to submit the render.com URL. Every member should submit a checklist of what you feel you have accomplished from the rubric above (including who did what), and include your total expected score. This can be as a text entry submission (if not submitting the URL), or as a comment once you submit the URL.

Groups

Section 01

Harrison, Erin, Kristina • TBD	Jesse, Samuel, Alejandro • TBD	Luke, Hunter, Zach • TBD
Vincent, Ada, Anne • TBD	Semone, Andres, Ian • TBD	Irqa, Angelika, Alexander • TBD
Keng, Kayla, Breanna • TBD	Sean, Ryan • TBD	Hadi, Lukas • TBD

Section 02

Emmanuel, Reece, Abdiwahab • TBD	Prince, Jair, Eli • TBD	Mark, Dylan G., Jack • TBD
Anastasia, Zoe, Carter • TBD	Ibrahima, Branden, Dylan T. • TBD	Akhil, Colton, Caiden • TBD
Cody, Charlie, Martin • TBD	Kino, Kayden, Charlotte • TBD	Jackson, Eyoel • TBD

Deadline

This assignment is due Thursday, October 30 at 10:00pm.